

Activity and Subject Detection for UCI HAR Dataset with & without missing Sensor Data

Debashish Saha
dept. of Electrical Eng.
Stony Brook University
Stony Brook, USA

debashish.saha@stonybrook.edu

Piyush Malik
dept. of Comp. Sci.
Stony Brook University
Stony Brook, USA
pimalik@cs.stonybrook.edu

Adrika Saha
dept. of Comp. Eng.
American International University Bangladesh
Dhaka, Bangladesh
18-37359-1@student.aiub.edu

Abstract—Current studies in Human Activity Recognition (HAR) primarily focus on the classification of activities through sensor data, while there is not much emphasis placed on recognizing the individuals performing these activities. This type of classification is very important for developing personalized and context-sensitive applications. Additionally, the issue of missing sensor data, which often occurs in practical situations due to hardware malfunctions, has not been explored yet. This paper seeks to fill these voids by introducing a lightweight LSTM-based model that can be used to classify both activities and subjects. The proposed model was used to classify the HAR dataset by UCI [1], achieving an accuracy of 93.89% in activity recognition (across six activities), nearing the 96.67% benchmark, and an accuracy of 80.19% in subject recognition (involving 30 subjects), thereby establishing a new baseline for this area of research. We then simulate the absence of sensor data to mirror real-world scenarios and incorporate imputation techniques, both with and without Principal Component Analysis (PCA), to restore incomplete datasets. We found that K-Nearest Neighbors (KNN) imputation performs the best for filling the missing sensor data without PCA because the use of PCA resulted in slightly lower accuracy. These results demonstrate how well the framework handles missing sensor data, which is a major step forward in using the Human Activity Recognition dataset for reliable classification tasks.

I. INTRODUCTION

Human Activity Recognition (HAR) has become a significant research area within the field of ambient and context-aware computing. Because of its many uses, such as healthcare monitoring, abnormal behavior detection, human-computer interaction, and assistive technologies to improve the quality of life for senior citizens, the ability to recognize daily activities is becoming more and more important in our everyday life. HAR frameworks use information gathered from several sensors to make it easier to detect, identify, and categorize particular human movements or activities.

The two main categories of HAR are wearable sensor-based and vision-based techniques. There are many challenges in using video sequences for human activity recognition, including privacy concerns, high computational complexity, and limited pervasiveness, which makes it difficult to capture full-body movements while moving around [2]. All of these concerns make it difficult to use vision-based sensors in practical situations.

On the other hand, wearable sensors and the inertial sensors included in our everyday smart gadgets provide a more useful and discrete way to gather information about human activities. These sensors require little to no installation work and are small, easy to use, economical, and energy-efficient. For HAR tasks, gadgets like smartphones and smartwatches with sensors like compasses, gyroscopes, magnetometers, and accelerometers have become quite practical [3]. It is still very difficult to generalize HAR models to different activities and sensor configurations. Because various people do the same action differently and because different people repeat the same activity at different times, activity signal patterns are very different. Recognizing the subject from the signal data is a difficult and complex undertaking since overlapping signal patterns between different activities make the classification procedure even more difficult.

Although deep learning-based Human Activity Recognition (HAR) has made great strides, current datasets and studies mostly focuses on activity classification rather than individual identification of the performers of those activities. This leaves a need for creating models capable of doing subject recognition, which could allow for more customized and context-aware applications. Furthermore, little study has been done on the problem of missing sensor data, a common occurrence in real-world settings because of hardware constraints, climatic conditions, or communication faults causing sensor failures. Missing data greatly affects the accuracy and dependability of the models, hence there is a need for managing insufficient sensor data. This work advances HAR research by suggesting a strong and scalable LSTM-based method able to provide dependable performance for both activity and subject recognition, even under missing sensor data.

II. RELATED WORK

HAR datasets have seen significant progress, especially in the area of deep learning-based activity classification. For example, Ronao and Cho's study [4] showed that Deep Convolutional and LSTM Recurrent Neural Networks may be used to recognize activities, with a **95.75%** accuracy rate on the UCI HAR dataset. A model based on Collaborative Edge Learning (CELearning) was also presented by Xu, Tang, Jin and Pan [5], and it attained an even better accuracy of **96.67%**. These

techniques demonstrate the effectiveness of multilayered deep learning models.

On the other hand, there is still much to learn about the field of subject recognition with HAR datasets. As of right now, there isn't a commonly used model or benchmark for subject classification utilizing the UCI HAR dataset. This disparity highlights the need for additional study in this field, especially for applications that call for context-aware algorithms and individualized insights.

Furthermore, one of the most important challenges for enhancing the resilience and dependability of HAR systems in practical contexts is dealing with missing sensor data. A unique method for improving activity detection without specifically recovering lost data was proposed by Hossain et al. [6]. Their approach boosted recognition accuracy by learning to handle partial datasets efficiently through the introduction of unpredictable missing data patterns throughout the training process. In a similar vein, Xavier et al. [7] created the Centaur multimodal fusion model, which used self-attention mechanisms to capture cross-sensor correlations and a denoising autoencoder in conjunction with convolutional layers to clean noisy data. The Centaur model maintained strong performance in HAR tasks despite noise and repeated missing data across various sensor modalities.

These experiments demonstrate some progress in handling incomplete data and activity recognition. Approaches that can handle both activity and subject detection are still needed, particularly in situations where data is missing. To address these issues we have created a lightweight LSTM-based system that can be used to classify either subjects or activities; and incorporate strong imputation techniques for handling imperfect sensor data.

III. METHODOLOGY

Our proposed framework uses the UCI Human Activity Recognition (HAR) dataset, which consists of data from 30 participants performing six different activities: *WALKING*, *WALKING_UPSTAIRS*, *WALKING_DOWNSTAIRS*, *SITTING*, *LAYING*, *STANDING*. Our goal is to create a model that can be used to train a classifier either for activity or the subject, even in the presence of missing sensor data.

The framework integrates three main components:

- Establish baseline accuracy with and without missing data using Long Short-Term Memory (LSTM) networks.
- Imputation techniques to handle missing sensor data, utilizing either a Univariate Imputer (`SimpleImputer`) or a K-Nearest Neighbors Imputer (`KNNImputer`).
- Dimensionality reduction using Principal Component Analysis (PCA) to simplify the data while retaining essential information.

A. LSTM Framework

We construct a simple Long Short-Term Memory (LSTM) network, as shown in Fig. 1, to serve as the baseline model. The network is defined as follows:

Let the input sensor data sequence be represented as $X = [x_1, x_2, \dots, x_T]$, where $x_t \in \mathbb{R}^d$ is the sensor data at time t , with d features. The Long Short-Term Memory (LSTM) processes this sequence by iteratively updating the hidden state h_t and cell state c_t at each time step using the following equations:

1. **Forget Gate**:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

2. **Input Gate**:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

3. **Output Gate**:

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

4. **Candidate Cell State**:

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

5. **Cell State Update**:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

6. **Hidden State Update**:

$$h_t = o_t \odot \tanh(c_t)$$

Here, σ is the sigmoid activation function, $\tanh$ is the hyperbolic tangent activation, and $\odot$ is the element-wise multiplication operation.

Dropout Layer To prevent overfitting, a dropout layer is applied to the output of the LSTM layer. This layer randomly sets a fraction p of the hidden weights to zero during training, making sure that the model does not become overly trained on the training data. Mathematically, the dropout operation for the LSTM's final output can be expressed as:

$$h_t^{\text{dropout}} = h_t \odot m, \quad m \sim \text{Bernoulli}(1 - p)$$

where m is a mask vector sampled from a Bernoulli distribution with probability $1 - p$, and p is the dropout rate.

Final Classification Layer After processing the entire sequence, the hidden state at the final time step, h_T^{dropout} , is passed through a fully connected layer followed by the softmax function to produce the predicted class probabilities:

$$y = \text{softmax}(W_y h_T^{\text{dropout}} + b_y)$$

where $y \in \mathbb{R}^K$ represents the probability distribution over K activity classes.

Loss Function The model is trained by minimizing the categorical cross-entropy loss, defined as:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k})$$

Here: - N is the number of samples in the training batch, - K is the number of classes, - $y_{i,k}$ is the true label (1 if sample i belongs to class k , and 0 otherwise), - $\hat{y}_{i,k}$ is the predicted probability for class k of sample i .

This loss function ensures that the model learns to maximize the predicted probabilities for the correct activity or subject class.

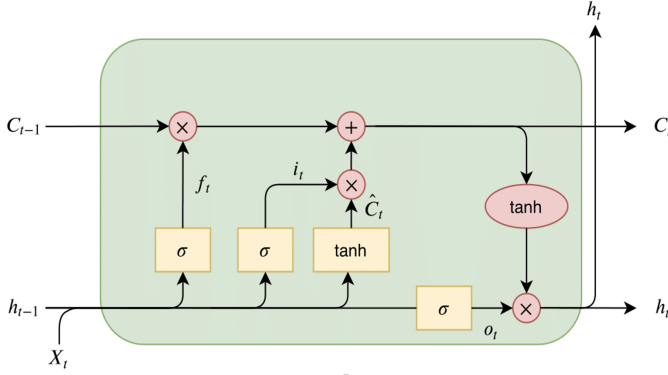


Fig. 1. LSTM Network Architecture

B. Handling Missing Sensor Data

To simulate real-world scenarios, missing data is introduced randomly in the dataset. The simulations introduced missing data for durations ranging from 1 to 10 seconds intervals assuming that the accelerometer and/or the gyroscope becomes non-functional. The missing data is imputed using the following methods:

Simple Imputation Missing values x_{ij} are replaced with the mean μ_j or median of the j^{th} feature:

$$x_{ij} = \mu_j = \frac{1}{N} \sum x_{ij} \quad (\text{if missing}) \quad (9)$$

KNN Imputation Missing values are estimated using the k -nearest neighbors, where the imputed value is:

$$x_{ij} = \frac{1}{k} \sum_{k \in \text{neighbors}} x_{ik} \quad (10)$$

C. Dimensionality Reduction with PCA

To reduce data dimensionality and noise, Principal Component Analysis (PCA) is applied. Given a dataset $X \in \mathbb{R}^{N \times d}$, PCA projects X onto r -dimensional space ($r < d$):

$$Z = XW \quad (11)$$

Here, W contains the eigenvectors of the covariance matrix of X , corresponding to the top r eigenvalues.

IV. DATASET AND EXPERIMENTS

For our experiment, we used the Human Activity Recognition Using Smartphones dataset by UCI [1], which includes data collected from 30 participants performing six activities: *WALKING*, *WALKING_UPSTAIRS*, *WALKING_DOWNSTAIRS*, *SITTING*, *LAYING*, and *STANDING*. The dataset comprises 10,299 entries and 561 features, divided into predefined training and testing sets. All the features were collected using only two sensors: the accelerometer and the gyroscope. Input features were normalized to ensure improved convergence and stability during model training.

The training was performed using an LSTM-based deep learning model implemented in Python. We used the Keras library with a TensorFlow backend. We designed a model architecture to balance simplicity and performance. The architecture consists of:

- An LSTM layer with 128 units, which processes the input sequences from the dataset.
- A dropout layer with a rate of 20%, to reduce overfitting by randomly deactivating neurons during training.
- Two dense layers:
 - The first dense layer has 64 units with ReLU activation to capture hierarchical feature representations.
 - The second dense layer applies a softmax activation function for multi-class classification.

The model was compiled using the categorical crossentropy loss function and the Adam optimizer, with accuracy used as the evaluation metric.

For activity recognition, the training was straightforward. We trained the LSTM model with the training data using a 20% validation split. For subject recognition, we had to modify the dataset. In the original dataset, there are a total of 30 subjects, but the training data includes only 21 subjects, and the test data includes 9 subjects. This distribution is not ideal for performing subject recognition. We needed a dataset where all 30 subjects were present in both the train and test sets. To achieve this, we combined the existing train and test sets into a single dataset and then used the `train_test_split` function provided by the `sklearn` library to split the data. Similar to the original training set, 20% of the data was reserved for our newly created test set.

We simulated scenarios with missing sensor data to evaluate the robustness of the proposed framework under real-world conditions where sensor failures are likely. The dataset, sampled at 50 Hz, contains 345 accelerometer features and 213 gyroscope features among its 561 total features. To mimic sensor failures, we introduced missing data for durations ranging from 1 to 10 seconds, simulating instances where either the accelerometer or gyroscope becomes non-functional. We envisioned 6 such scenarios, and in each scenario, a total of 10 seconds of data was missing, which meant that approximately 24.27% of rows in the dataset contained some missing sensor values.

To address these missing values, we applied `SimpleImputer` and `KNNImputer` techniques, both with and without Principal Component Analysis (PCA). PCA was applied for dimensionality reduction, with 175 components retained. This number was determined based on our analysis, which showed that the retention of 175 components was necessary to preserve 99% of the variance in the dataset.

V. EVALUATION

We conducted training and testing for both activity and subject recognition. We trained the model for 300 epochs, and used the `Accuracy` score of the test set for our metrics.

The LSTM-based model was evaluated on the HAR dataset without missing sensor data for activity recognition. It achieved a test accuracy of **93.89%** for classifying six activities: *WALKING*, *WALKING_UPSTAIRS*, *WALKING_DOWNSTAIRS*, *SITTING*, *LAYING*, *STANDING*. It is very close to the benchmark accuracy of 96.67% achieved by more complex Collaborative Edge Learning (CELearning) models [5].

For subject recognition, our model achieved a test accuracy of **80.19%** for classifying 30 subjects. Currently there is no benchmark for subject recognition for this dataset. If we look at the subject class accuracy per activity, the picture becomes much clearer. Among all subjects, it is the *SITTING*, *LAYING*, *STANDING* activities that cause problems. Since these are stationary activities, they might appear very similar across all subjects, as shown in Figure 2.

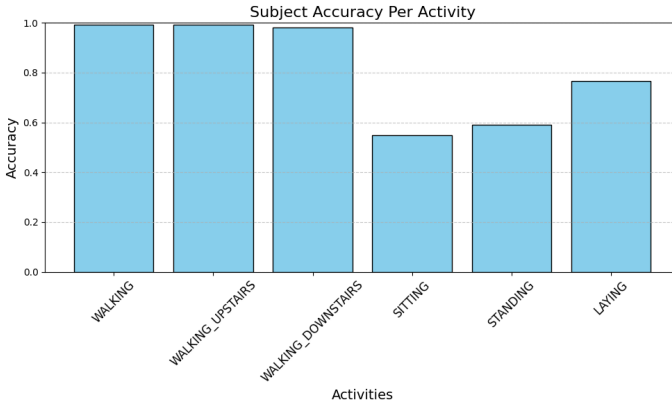


Fig. 2. Subject recognition accuracy per activity

The results demonstrate the efficiency of our architecture, which uses an LSTM layer, dropout for regularization, and dense layers for classification.

A. Insights from Missing Data Experiments

The following six scenarios were created by us to evaluate the impact of missing sensor data on the proposed framework:

- 1) **1 Second ACC & 1 Second Gyro (5 intervals):**
 - A total of 5 intervals, each lasting 1 second, where both accelerometer (ACC) and gyroscope (Gyro) data are missing.
 - Proportion of missing data: **12.07%**.
- 2) **5 Second ACC & 5 Second Gyro (1 interval):**
 - A single 5-second interval where both accelerometer and gyroscope data are missing.
 - Proportion of missing data: **12.06%**.
- 3) **5 Second ACC (2 intervals):**
 - Two intervals, each lasting 5 seconds, where only accelerometer (ACC) data is missing.
 - Proportion of missing data: **14.91%**.
- 4) **5 Second Gyro (2 intervals):**
 - Two intervals, each lasting 5 seconds, where only gyroscope (Gyro) data is missing.

- Proportion of missing data: **9.22%**.

5) **10 Seconds ACC (1 interval):**

- A single 10-second interval where only accelerometer (ACC) data is missing.
- Proportion of missing data: **14.91%**.

6) **10 Seconds Gyro (1 interval):**

- A single 10-second interval where only gyroscope (Gyro) data is missing.
- Proportion of missing data: **9.22%**.

VI. RESULTS

The results of the missing data experiments are promising. The baseline **activity recognition** accuracy of the model was **93.85%** under ideal conditions with no missing data. When we introduced missing sensor data, accuracy of the model reduced significantly. With the lowest accuracy dropping to **77.29%** depending on the scenario. For **subject recognition** the accuracy was **80.19%** with no missing data and with missing data it goes as low as **61.35%**. Applying imputation techniques showed substantial recovery in performance as shown in Table I and Table II.

TABLE I
ACCURACY WITH SIMULATED MISSING DATA AND IMPUTATION FOR ACTIVITY RECOGNITION

Missing data intervals	Accuracy with missing data	SI	KNN	SI+PCA	KNN+PCA
1s ACC 1s Gyro (5 intervals)	0.7822	0.8989	0.8860	0.8975	0.8856
5s ACC 5s Gyro (1 interval)	0.7872	0.8901	0.8931	0.8907	0.8928
5 Sec ACC (2 intervals)	0.7716	0.8663	0.9233	0.8653	0.9237
5s Gyro (2 intervals)	0.7974	0.9189	0.9301	0.9186	0.9284
10s ACC (1 intervals)	0.7710	0.8626	0.9138	0.8612	0.9138
10s Gyro (1 interval)	0.7740	0.9308	0.9362	0.9298	0.9348

TABLE II
ACCURACY WITH SIMULATED MISSING DATA AND IMPUTATION FOR SUBJECT RECOGNITION

Missing data intervals	Accuracy with missing data	SI	KNN	SI+PCA	KNN+PCA
1s ACC & 1s Gyro (5 intervals)	0.6150	0.6417	0.7340	0.6282	0.7199
5s ACC & 5s Gyro (1 interval)	0.6184	0.6383	0.7340	0.6262	0.7170
5s ACC (2 intervals)	0.6180	0.6413	0.7442	0.6277	0.7282
5s Gyro (2 intervals)	0.6141	0.6510	0.7515	0.6403	0.7398
10s ACC (1 interval)	0.6155	0.6340	0.7476	0.6233	0.7340
10s Gyro (1 interval)	0.6170	0.6490	0.7587	0.6379	0.7481

From the data above we gathered couple of key insights

- **Simple Imputer:** Achieved accuracy recovery across all scenarios, ranging from **86.26%** to **93.08%** for activity

recognition and **63.40%** to **64.90%** for subject recognition.

- **KNN Imputer:** Consistently outperformed or got very close to Simple Imputer in most scenarios, achieving accuracy ranging from **88.60%** to **93.62%** for activity recognition (without missing data accuracy was **93.89%**) and **73.40%** to **75.87%** for subject recognition (without missing data accuracy was **80.19%**).

Combining PCA with imputation provided mixed results. While PCA reduced dimensionality, it did not consistently improve accuracy. In the majority of cases, accuracy is slightly worse. For example:

- **Simple Imputer with PCA:** Simple Imputer with PCA has an accuracy score ranging from **86.12%** to **92.98%** for activity recognition. For subject recognition, it ranges from **62.33%** to **64.03%**.
- **KNN Imputer with PCA:** KNN Imputer with PCA has an accuracy score ranging from **88.56%** to **93.48%** for activity recognition. For subject recognition, it ranges from **71.99%** to **74.81%**.

The data also revealed that missing gyroscope data is easier to recover compared to accelerometer data. If we do a bar graph with all the accuracy metrics, the overall picture becomes much clearer. Below, you can see Figure 3 for activity recognition and Figure 4 for subject recognition.

In the bar graphs:

- **Red Dotted Line:** Baseline Accuracy (without missing data, ideal case).
- **Blue Bar:** Accuracy with Missing Data (no imputation applied).
- **Orange Bar:** Accuracy after applying **Simple Imputer**.
- **Green Bar:** Accuracy after applying **KNN Imputer**.
- **Red Bar:** Accuracy after applying **Simple Imputer + PCA**.
- **Purple Bar:** Accuracy after applying **KNN Imputer + PCA**.

As you can see all imputation techniques perform better than just with missing data and gets very close to the accuracy without missing data. These visualizations shows the effectiveness of imputation techniques and the recovery of accuracy for both activity and subject recognition tasks. This highlights the remarkable effectiveness of the KNN Imputer in handling missing sensor data independently without PCA. Our framework's ability to recover accuracy demonstrates its potential for real-world applications where sensor failures are very common.

VII. CONCLUSION

Our study advances the field of Human Activity Recognition by addressing two critical gaps: identifying subjects with HAR sensor data, and the ability to handle missing sensor data. By leveraging an lightweight LSTM-based model and simulating missing data, something that happens in real world quiet often, we demonstrated the robustness and adaptability of our framework. We have established a new benchmark for subject recognition for HAR dataset and we showed that both activity and

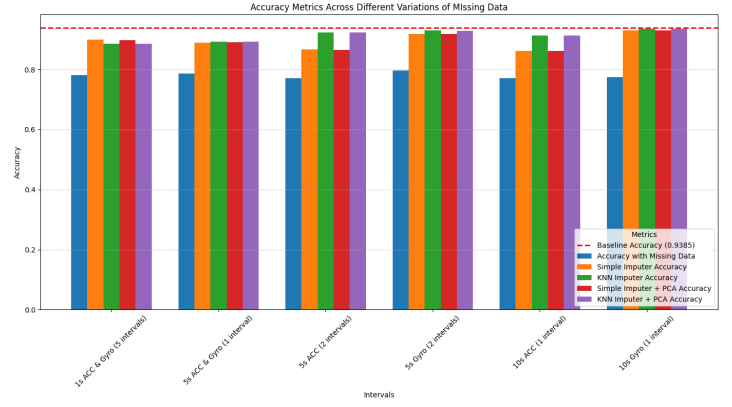


Fig. 3. Accuracy for **activity recognition** with Imputations. The blue bars represent baseline results with missing data, and the red dotted line indicates results without missing data.

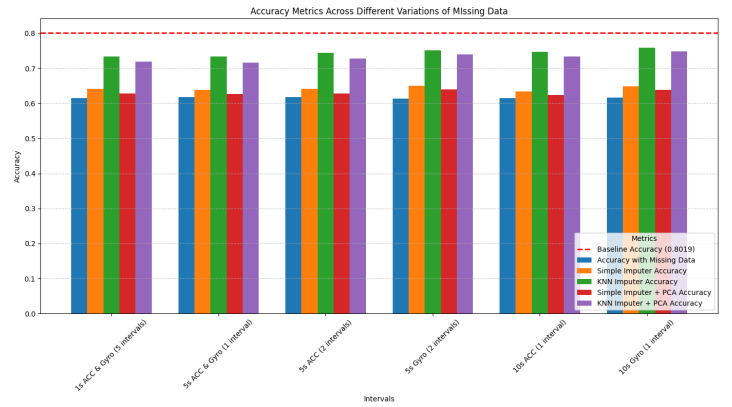


Fig. 4. Accuracy for **subject recognition** with Imputations. The blue bars represent baseline results with missing data, and the red dotted line indicates results without missing data.

subject recognition can be easily done even with incomplete sensor data, using imputation techniques like KNN Imputer. Our work provides a foundation for creating more personalized and reliable HAR applications. Future work may explore extending our framework and try more advanced imputation methods to create a more robust classifier. The source code for this framework is publicly available at https://github.com/debashish-saha1/HAR_UCI_DATASET_MISSING_DATA.

REFERENCES

- [1] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "Human Activity Recognition Using Smartphones [Dataset]. UCI Machine Learning Repository," 2013. doi: <https://doi.org/10.24432/C54S4K>.
- [2] O. D. Lara and M. A. Labrador, "A Survey on Human Activity Recognition using Wearable Sensors," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1192–1209, 2013. doi: <https://doi.org/10.1109/SURV.2012.110112.00192>.
- [3] M. Kaseris, I. Kostavelis, and S. Malassiotis, "A Comprehensive Survey on Deep Learning Methods in Human Activity Recognition," *Machine learning and knowledge extraction*, vol. 6, no. 2, pp. 842–876, Apr. 2024. doi: <https://doi.org/10.3390/make6020040>.
- [4] C. A. Ronao and S.-B. Cho, "Human activity recognition with smartphone sensors using deep learning neural networks," *Expert*

Systems with Applications, vol. 59, pp. 235–244, Oct. 2016, doi:
<https://doi.org/10.1016/j.eswa.2016.04.032>.

- [5] S. Xu, Q. Tang, L. Jin, and Z. Pan, “A Cascade Ensemble Learning Model for Human Activity Recognition with Smartphones,” *Sensors*, vol. 19, no. 10, p. 2037, May 2019, doi:
<https://doi.org/10.3390/s19102307>.
- [6] T. Hossain, Md. A. R. Ahad, and S. Inoue, “A Method for Sensor-Based Activity Recognition in Missing Data Scenario,” *Sensors*, vol. 20, no. 14, pp. 3811, Jul. 2020, doi:
<https://doi.org/10.3390/s20143811>.
- [7] S. Xaviar, X. Yang, and O. Ardakanian, “Robust Multimodal Fusion for Human Activity Recognition,” *arXiv.org*, 2023, doi:
<https://doi.org/10.48550/arXiv.2303.04636>.